Guia para Apresentação — PF4 e PF5

> *O Bestiário do Samuel · Grupo 7*

O que é a ferramenta?

Uma criança desenha um bicho no papel, tira uma foto com o tablet e conta falando como ele é. A IA transcreve a fala, interpreta o desenho, e gera três sprites pixel art diferentes para a criança escolher. O bicho escolhido entra no bestiário (uma coleção de cartas), ganha vida no zoológico digital (um cenário pixel art onde as criaturas andam e conversam entre si) e vira uma carta colecionável estilo retro para imprimir.

Para quem é?

Samuel, 8 anos, primo do Marcos. Joga Minecraft, inventa mundos, desenha bichos no papel, escreve pouco. A interface funciona toda por ícones, voz e toque. Sem texto, sem login, sem conta.

Como funciona o fluxo técnico?

1. **Foto** — a câmera do dispositivo captura o desenho. A imagem é reduzida para max 400px e comprimida (JPEG quality 0.5, max 300KB) para funcionar em qualquer rede e dispositivo.
2. **Gravação** — o microfone grava a descrição da criança. O formato de áudio é detectado automaticamente (webm no Android, mp4 no iOS, ogg como fallback).
3. **Transcrição** — o áudio é enviado para `gpt-transcribe` (OpenAI) com `language: "pt"` para otimizar reconhecimento de voz de criança em português.
4. **Interpretação** — a foto + texto são enviados para `gpt-6-luna` (modelo multimodal da OpenAI) que retorna 3 versões da criatura com nome, corpo, cores, comportamentos, bioma, origem, habilidade, curiosidade e um prompt para gerar o sprite.
5. **Geração de sprites** — 3 chamadas paralelas para `gpt-image-1-mini` (OpenAI) geram sprites pixel art 1024x1024 em WebP com fundo transparente. Cada sprite recebe uma combinação aleatória de estilo, paleta de cor e detalhe (640 combinações possíveis).
6. **Escolha** — a criança vê as 3 opções com sprite + nome + descrição + atributos e toca na que mais gosta.
7. **Celebração** — confetti CSS + voz "X nasceu!" + animação de entrada.
8. **Bestiário** — coleção de cartas com sprites pixel art.
9. **Zoológico** — cenário pixel art gerado por IA onde os sprites andam e conversam automaticamente quando se encontram.
10. **Carta** — card colecionável estilo retro com origem, habilidade, curiosidade e comportamentos.
11. **História** — IA gera narrativa de 8-12 frases com diálogo e arco narrativo, com opção de leitura em voz alta.

Quais tecnologias foram usadas?

Camada	Tecnologia	Para quê
Frontend	Next.js 15 + React 19 + TypeScript	App web single-page, tablet-first
Ícones	Lucide React	Ícones padronizados em todos os botões
Estilos	CSS puro (sem Tailwind)	Animações, cenário pixel art, cartas retro
Transcrição	OpenAI gpt-transcribe	Fala da criança em português para texto

Interpretação	OpenAI gpt-6-luna	Modelo multimodal que lê imagem + texto
Geração de sprites	OpenAI gpt-image-1-mini	Pixel art 1024x1024, fundo transparente, WebP
Voz (TTS)	OpenAI gpt-4o-mini-tts (voz marin)	Instruções faladas para a criança
Histórias	OpenAI gpt-6-luna	Narrativa infantil a partir dos dados da criatura
Diálogos	OpenAI gpt-6-luna	Conversas entre criaturas no zoológico
Armazenamento	localStorage (navegador)	Bestiário salvo localmente, sem servidor
Hospedagem	Vercel	Deploy automático, HTTPS, serverless functions
Segurança	Rate limit in-memory (30 req/min/IP)	Proteção contra abuso das APIs

---

Quais modelos de IA foram usados e para quê?

Modelo	Provedor	Uso	Custo por criatura
gpt-transcribe	OpenAI	Transcrição de áudio para texto (STT)	~US$ 0,006
gpt-6-luna	OpenAI	Interpretação multimodal (imagem + texto), geração de histórias e diálogos	~US$ 0,03
gpt-image-1-mini	OpenAI	Geração de sprites pixel art (3 por criatura)	~US$ 0,12 (3x US$ 0,04)
gpt-4o-mini-tts	OpenAI	Voz das instruções e leitura de histórias	~US$ 0,01
Total por criatura			**~US$ 0,17**

Onde a IA entra e o que ela decide?

A IA entra em 5 momentos:

1. **Transcrição** — converte a fala da criança em texto. Sem isso, a criança precisaria digitar.
2. **Interpretação** — analisa o desenho junto com a descrição e propõe 3 versões da criatura com personalidade, bioma e atributos. Sem isso, não haveria interpretação automática.
3. **Geração visual** — cria sprites pixel art únicos para cada versão. Sem isso, o grupo precisaria desenhar manualmente.
4. **Voz** — fala as instruções e narra as histórias. Sem isso, a criança precisaria ler.
5. **Narrativa e diálogos** — cria histórias e conversas entre criaturas com personalidade baseada nos comportamentos.

Quem decide é sempre a criança. A IA propõe, ela escolhe.

Quais problemas foram enfrentados?

Problema 1: Fotos grandes demais no celular

O que acontecia: A câmera do celular captura fotos em alta resolução (ex: 4032x3024, ~5MB). Ao converter para base64 e enviar no body JSON, o payload excedia o limite de 4.5MB da Vercel. A rota `/api/interpretar` falhava silenciosamente e o app mostrava "bestiário está dormindo".

Como resolvemos:

- A câmera pede resolução 640x480 (não a nativa).
- A foto é redimensionada para max 400px no lado maior.
- Compressão JPEG iterativa: começa em quality 0.5 e reduz até a foto ficar abaixo de 300KB.
- No momento do envio, se ainda estiver grande, recomprime para 320px quality 0.4 como safety net.
- A IA não precisa de alta resolução para interpretar um desenho infantil.

Problema 2: Formato de áudio varia entre dispositivos

O que acontecia: O `MediaRecorder` grava em formatos diferentes dependendo do dispositivo: `audio/webm;codecs=opus` no Chrome/Android, `audio/mp4` no Safari/iOS, `audio/ogg` em Firefox. O arquivo era sempre nomeado como `fala.webm`, e a API de transcrição às vezes rejeitava.

Como resolvemos:

- Antes de criar o MediaRecorder, testamos quais formatos são suportados pelo dispositivo.
- A extensão do arquivo é escolhida com base no mime type real (`.webm`, `.mp4`, `.ogg`).
- Se nenhum formato específico é suportado, usa o MediaRecorder sem configuração de mime type.

Problema 3: Modelo de geração de imagem depreciado

O que acontecia: Inicialmente usamos `dall-e-3`, que retornava erro "The model 'dall-e-3' does not exist." O modelo foi depreciado e substituído pela família `gpt-image-*`.

Como resolvemos:

- Migramos para `gpt-image-1-mini` (mais rápido e barato).
- Adaptamos o formato de resposta: `gpt-image` retorna `b64_json` direto em vez de URL temporária.
- Mudamos para WebP com fundo transparente (489KB vs 1.4MB em PNG).

Problema 4: Tamanho de imagem não suportado

O que acontecia: Tentamos gerar sprites em `256x256` para serem menores, mas a API retornou "Invalid size '256x256'". O modelo só aceita `1024x1024`, `1024x1536`, `1536x1024` e `auto`.

Como resolvemos:

- Geramos em `1024x1024` (menor tamanho aceito).
- Usamos `output_format: "webp"` que comprime melhor que PNG (~500KB vs 1.4MB).
- O pixel art fica bom em qualquer tamanho porque o estilo já é pixelado.

Problema 5: Sprites repetitivos e previsíveis

O que acontecia: As 3 opções de sprite para o mesmo desenho eram visualmente muito parecidas. A criança não via diferença real entre as opções.

Como resolvemos:

- Criamos um sistema de aleatoriedade com 3 dimensões: 10 estilos (chibi, fierce, sleepy, ghostly...) x 8 paletas de cor (neon, sunset, pastel, earthy...) x 8 detalhes (sparkles, companion, glowing eyes...) = **640**

combinações possíveis.

- Cada sprite recebe uma combinação aleatória diferente.
- Mesmo desenho gera 3 sprites visualmente distintos.

Problema 6: Tempo de carregamento dos sprites

O que acontecia: Gerar 3 sprites leva ~30 segundos no total. Se bloqueávamos a tela esperando, a criança perdia interesse. Se mostrávamos sem sprites, não fazia sentido escolher.

Como resolvemos:

- Mantemos a tela de loading com ovo animado que evolui em 5 fases: balança lento, balança rápido, rachaduras aparecem, mais rachaduras com glow, brilho dourado pulsante.
- Cada fase tem uma mensagem diferente narrada pela voz ("Escutando o que você falou...", "Olhando o seu desenho...", "Inventando três versões...", "Pintando as cores de cada um...", "Quase lá!").
- Dots de progresso no rodapé.
- A escolha só aparece quando as 3 sprites estão prontas.

Como é a experiência para a criança?

1. **Tutorial** — na primeira vez, 4 slides animados explicam o que fazer (desenha, conta, escolhe, zoológico). Pode pular.
2. **Desenha** — pega papel e canetinha e desenha o bicho.
3. **Fotografa** — posiciona o desenho na câmera e toca no botão.
4. **Fala** — segura o botão do microfone e descreve o bicho ("ele cospe fogo, usa capa e é colorido").
5. **Espera** — ovo animado racha enquanto a IA trabalha (~30-40s).
6. **Escolhe** — 3 sprites pixel art diferentes. Toca no favorito.
7. **Celebra** — confetti + voz "X nasceu!" + animação.
8. **Explora** — vê o detalhe, a carta retro, a história, ou vai pro zoológico.
9. **Zoológico** — os bichos andam pelo cenário pixel art. Quando dois se encontram, param e conversam automaticamente com balões de fala.
10. **Cria mais** — repete o processo para encher o bestiário.

Como roda localmente?

```
git clone https://github.com/marcosdidier/criacomp-ferramenta-grupo7.git
cd criacomp-ferramenta-grupo7
npm install
cp .env.example .env.local
# Edite .env.local com OPENAI_API_KEY
npm run dev
# Abra http://localhost:3000
```

Quanto custa rodar?

Para 50 criaturas (testes + demonstração + Avaliação Cruzada): **~US$ 8,50**.

A maior parte do custo é a geração de sprites (3 por criatura x US$ 0,04 cada).

E a privacidade?

- Samuel é menor de idade. Os pais autorizaram os testes (17/10/2026).
- Sem login, sem conta, sem dados pessoais.
- O áudio é enviado para transcrição e não é armazenado no servidor.
- A foto é enviada para interpretação e não é armazenada no servidor.
- Tudo fica em localStorage no navegador da criança.
- O servidor é stateless — não persiste nenhum dado.
- As chaves de API ficam nas variáveis de ambiente da Vercel.
- Rate limit de 30 requisições por minuto por IP protege contra abuso.

Na Avaliação Cruzada, como funciona sem o grupo?

Tudo roda no navegador. O outro grupo abre https://criacomp-ferramenta-grupo7.vercel.app, passa pelo tutorial de 4 slides, desenha, fala, escolhe e vê o bicho no zoológico. Sem instalar nada, sem explicação.

Links

- **App:** https://criacomp-ferramenta-grupo7.vercel.app
- **Repositório:** https://github.com/marcosdidier/criacomp-ferramenta-grupo7
- **Arquitetura:** `docs/1-Entregas/PF3-Diagrama-Arquitetura.md`
- **Protótipo original:** `docs/1-Entregas/PF2-Prototipo-Baixa-Fidelidade.md`